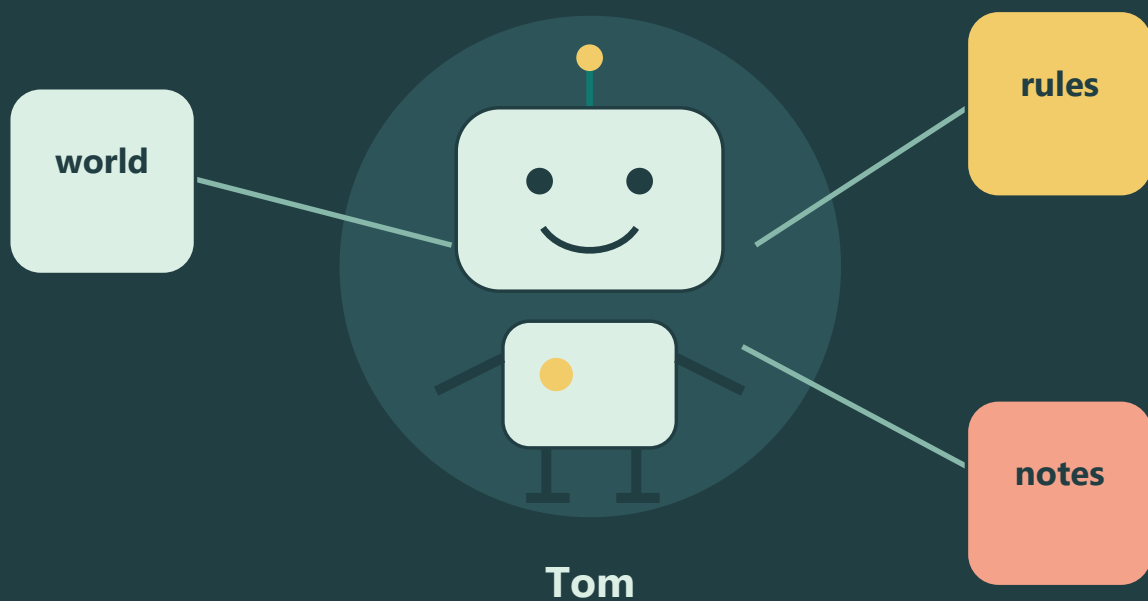


# A little machine with a world of its own

Tom Klootwijk's idea, told with rooms, rule cards and a very small robot.



**Imagine one little machine whose world, instructions and memories use the same kind of small building block.**

Its current situation helps choose its next step. It can even rebuild details it made from rules and put away, using the original recipe and the notes that recipe needs.

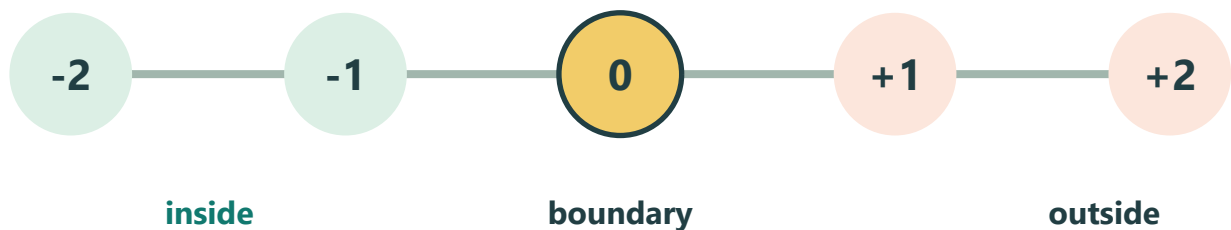
Our robot stands for a computer program. This is a story about how it works.

# Start with connections

Imagine a few rooms joined by paths. The machine describes its world through those connections: which rooms meet, and how long each path is.

Now mark a boundary. Every room gets a number telling us the shortest distance to it, following the paths. The size of the number tells us how far away; its sign tells us which side.

A tiny example: each path below is one step.



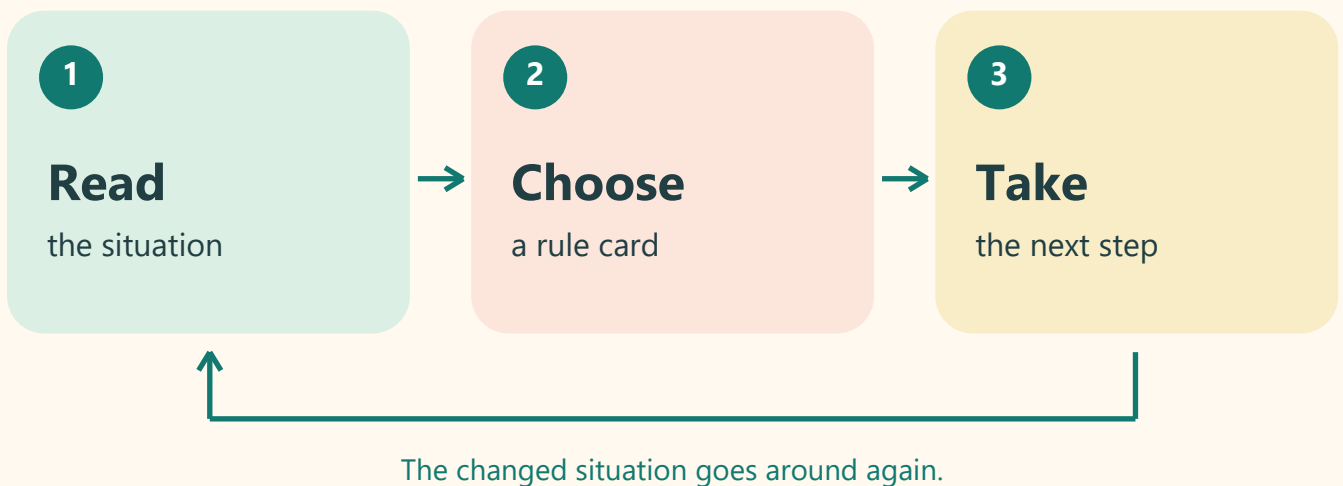
**Negative means inside. Zero means on the boundary. Positive means outside.**

The boundary could be the edge of a region in a little simulated world. The number gives the machine something exact to work with when choosing a rule.

**The connections and boundary give this world its shape.**

# Each step helps choose the next one

The machine reads its current situation and uses it to choose a rule card. Following that card changes the situation. That new situation helps choose the next card.



**Its own latest situation helps choose what happens next.**

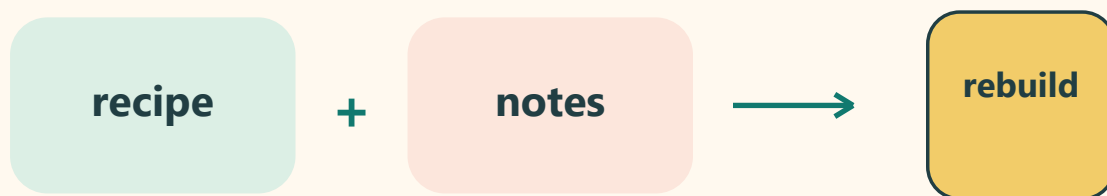
World facts, instructions and records all use the same kind of little block. Labels tell the machine how to read each one, like a card that belongs to a particular part of a game.

The rules are supplied. The loop follows them; it does not have to invent them as it goes.

# Remember the recipe

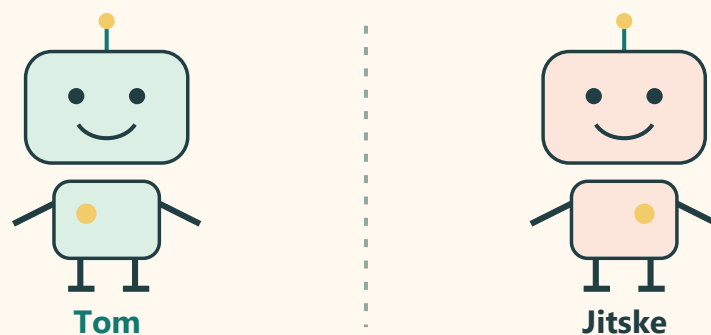
Suppose you build a toy house, then clear it off the table. You can rebuild it if you keep the instructions and every piece or note those instructions need.

The machine can do this with generated details of its world. A small worktable still needs somewhere to keep those recipes and notes.



## Two views, one individual

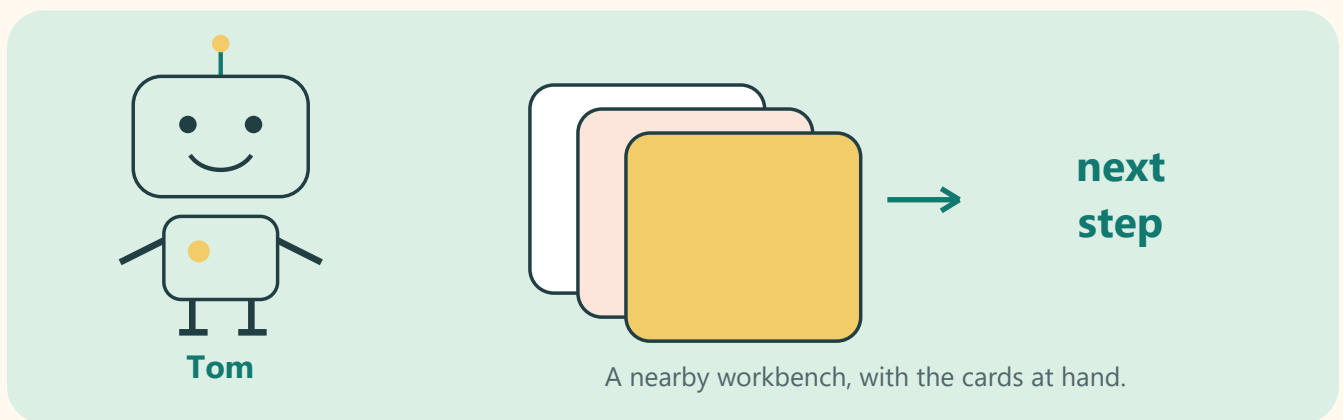
It also keeps a mirrored view of its state. Think of looking at the same toy from two sides. The views must agree in the way the rules say they should.



That helps spot certain mistakes. It is one individual, with two related views.

# Keep the cards close

The GPU is another kind of computer worker. It can keep the rule cards and the current state nearby, then follow a run of steps before checking back with the main computer.



## What “infallible” means here

Think of a game with a complete rulebook. Start from an allowed position, give the same information in the same order, and follow the rules faithfully.

**The same steps give the same answer. Any promise we have proved the rules keep stays true along the way.**

That promise depends on those conditions. It does not make faulty hardware or a wrong rule disappear.

# Several pieces already work

The small building blocks work. The machine can calculate exact boundary distances in its little connected worlds. The main computer and GPU can follow the same sequence and agree, step for step.

**262**

checks passed

**21**

ran on the GPU

**64**

matching steps

These are recorded checks from 25 September 2026. The GPU checks are part of the 262, and the 64 steps are one matching CPU/GPU example.

## The next pieces of the story

The special twisting world called a Klein bottle still needs to be built into the program. The world-building, memory and decision-making pieces also need to be joined into one evolving individual.

**The idea we are building toward:  
one individual with a world made of shared blocks,  
which it can explore and rebuild from saved recipes.**

Architecture by Tom Klootwijk. A friendly companion to The Infallible Contract (TK-LPLUT-2.0). The robot and rooms are explanatory pictures.